Prerequis

1. Installer pandas, sqlalchemy et pyodbc

```
pip install pandas sqlalchemy pyodbc
```

Sur Jupyter

```
%pip install pandas sqlalchemy pyodbc
```

Idée générale :

- **Extract** : on écrit les requêtes avec le **query builder** sqlalchemy (`select`, `where`, `join`, ...) et on récupère le résultat dans un **DataFrame** pandas.
- **Transform** : on nettoie / calcule / agrège avec pandas.
- **Load** : on renvoie le DataFrame dans le DW (query builder `insert`, ou `to_sql`).

Se connecter

2. Créer un moteur par base (la source et le DW)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

# mssql+pyodbc://username:password@localhost:1433/nom_de_la_db
url_source = "mssql+pyodbc://sa:test1234=@localhost:1433/GlacierApp"
url_source += "?driver=ODBC+Driver+17+for+SQL+Server"

url_dw = "mssql+pyodbc://sa:test1234=@localhost:1433/GlacierDW"
url_dw += "?driver=ODBC+Driver+17+for+SQL+Server"

engine_source = create_engine(url_source)
engine_dw = create_engine(url_dw, fast_executemany=True)   # insertions en masse
plus rapides

session_source = sessionmaker(bind=engine_source)
session_dw = sessionmaker(bind=engine_dw)
```

Les modèles (classes)

Comme d'habitude, une classe par table. Exemple pour la source :

```python
from datetime import datetime
from decimal import Decimal

from sqlalchemy import ForeignKey
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class BaseSource(DeclarativeBase):
    pass

class Produit(BaseSource):
    __tablename__ = "produit"
    produit_id: Mapped[int] = mapped_column(primary_key=True)
    code: Mapped[str] = mapped_column()
    nom: Mapped[str] = mapped_column()
    categorie_id: Mapped[int | None] =
mapped_column(ForeignKey("categorie.categorie_id"))
    prix_unitaire: Mapped[Decimal] = mapped_column()
    actif: Mapped[bool] = mapped_column()

class Commande(BaseSource):
    __tablename__ = "commande"
    commande_id: Mapped[int] = mapped_column(primary_key=True)
    magasin_id: Mapped[int] = mapped_column()
    client_id: Mapped[int | None] = mapped_column()
    date_commande: Mapped[datetime] = mapped_column()
    statut: Mapped[str] = mapped_column()
    mode_paiement: Mapped[str] = mapped_column()

class LigneCommande(BaseSource):
    __tablename__ = "ligne_commande"
    ligne_id: Mapped[int] = mapped_column(primary_key=True)
    commande_id: Mapped[int] = mapped_column(ForeignKey("commande.commande_id"))
    produit_id: Mapped[int] = mapped_column(ForeignKey("produit.produit_id"))
    quantite: Mapped[int] = mapped_column()
    prix_unitaire: Mapped[Decimal] = mapped_column()
    remise_pct: Mapped[Decimal] = mapped_column()

# + Categorie, Magasin ...
```

Et pour le DW (une **autre** classe de base, puisque c'est une autre base) :

```python
class BaseDW(DeclarativeBase):
    pass

class DimProduit(BaseDW):
    __tablename__ = "dim_produit"
    produit_key: Mapped[int] = mapped_column(primary_key=True)   # IDENTITY
    produit_id: Mapped[int] = mapped_column()
    code: Mapped[str] = mapped_column()
```

```
    nom: Mapped[str] = mapped_column()
    categorie: Mapped[str] = mapped_column()
    type_produit: Mapped[str] = mapped_column()
    prix_catalogue: Mapped[Decimal] = mapped_column()
    allergenes: Mapped[str | None] = mapped_column()
    est_vegan: Mapped[bool | None] = mapped_column()
    est_actif: Mapped[bool] = mapped_column()

# + DimMagasin, FaitVente ...
```

Lire des données (Extract)

Du query builder vers un DataFrame

`pd.read_sql` accepte directement une requête construite avec `select` :

```
import pandas as pd
from sqlalchemy import select

query = select(Produit.produit_id, Produit.code, Produit.nom,
Produit.categorie_id, Produit.prix_unitaire)

df_produits = pd.read_sql(query, engine_source)
print(df_produits.head())
```

Chaque élément du `select` devient une **colonne** du DataFrame (avec le même nom).

Pour toutes les colonnes d'une table, on passe simplement la classe :

```
df_produits = pd.read_sql(select(Produit), engine_source)
#    produit_id    code             nom  categorie_id  prix_unitaire  actif
# 0           1  GL-VAN  Cornet vanille           1.0           2.80   True
# ...
```

> Remarque : `categorie_id` contient des NULL → pandas le transforme en `float` (`1.0`, `NaN`). Voir « Pièges fréquents ».

Avec une session

Si on travaille déjà avec une session, le plus simple est de donner sa connexion à pandas :

```
with session_source() as session:
    df_produits = pd.read_sql(select(Produit), session.connection())
```

On peut aussi passer par `session.execute(...)`, mais **uniquement avec une liste de colonnes** :

```
with session_source() as session:
    query = select(Produit.produit_id, Produit.nom)
    result = session.execute(query)
    df_produits = pd.DataFrame(result.mappings().all())   # OK : une colonne par
attribut
```

`.mappings()` renvoie chaque ligne sous forme de dictionnaire `{colonne: valeur}` → pandas en fait directement un tableau.

> Attention : `session.execute(select(Produit))` renvoie des **objets** `Produit` (c'est l'ORM). Avec `.mappings()` on obtient `{"Produit": <objet>}`, donc un DataFrame avec **une seule colonne** remplie d'objets. Pour la classe entière, utiliser `pd.read_sql(...)`.

Avec deux classes dans le même `select`, les colonnes de même nom sont renommées automatiquement (`nom`, `nom_1`) : mieux vaut alors lister les colonnes et utiliser `.label()` (voir plus bas).

Filtrer et joindre

```
query = (
    select(Commande.commande_id,
           Commande.magasin_id,
           Commande.date_commande,
           LigneCommande.produit_id,
           LigneCommande.quantite,
           LigneCommande.prix_unitaire,
           LigneCommande.remise_pct)
    .join(LigneCommande, LigneCommande.commande_id == Commande.commande_id)
    .where(Commande.statut == "PAYEE")
    .where(Commande.magasin_id != 99)
)

df_ventes = pd.read_sql(query, engine_source)
```

Renommer une colonne : `.label()`

Utile quand deux tables ont une colonne du même nom :

```
query = (
    select(Produit.produit_id,
           Produit.nom,
           Categorie.nom.label("categorie"))      # sinon 2 colonnes "nom" !
    .join(Categorie, Categorie.categorie_id == Produit.categorie_id, isouter=True)
# LEFT JOIN
```

```
)
df_produits = pd.read_sql(query, engine_source)
```

`isouter=True` = **LEFT JOIN** : indispensable ici, sinon les produits sans catégorie disparaissent.

Agréger directement en SQL : `func` et `group_by`

On peut faire l'agrégation dans la requête plutôt qu'en pandas (moins de données transférées) :

```
from sqlalchemy import func, cast, Date

jour = cast(Commande.date_commande, Date).label("jour")

query = (
    select(jour,
           Commande.magasin_id,
           LigneCommande.produit_id,
           func.sum(LigneCommande.quantite).label("quantite"),
           func.sum(LigneCommande.quantite *
LigneCommande.prix_unitaire).label("montant_brut"),
           func.count(Commande.commande_id.distinct()).label("nb_commandes"))
    .join(LigneCommande, LigneCommande.commande_id == Commande.commande_id)
    .where(Commande.statut == "PAYEE", Commande.magasin_id != 99)
    .group_by(cast(Commande.date_commande, Date), Commande.magasin_id,
LigneCommande.produit_id)
)

df_agg = pd.read_sql(query, engine_source)
```

Voir le SQL généré

Pratique pour comprendre ou déboguer :

```
print(query)
```

Lire par morceaux (gros volumes)

```
for chunk in pd.read_sql(query, engine_source, chunksize=10_000):
    print(len(chunk))   # chaque chunk est un DataFrame de 10 000 lignes max
```

Transformer (Transform) avec pandas

```
# nettoyer du texte
df_magasins["ville"] = df_magasins["ville"].str.strip().str.title()

# remplacer les valeurs manquantes
df_produits["categorie"] = df_produits["categorie"].fillna("Non classé")

# date -> date_key AAAAMMJJ (int)
df_agg["jour"] = pd.to_datetime(df_agg["jour"])
df_agg["date_key"] = df_agg["jour"].dt.strftime("%Y%m%d").astype(int)

# colonne calculée
df_ventes["montant_brut"] = df_ventes["quantite"] * df_ventes["prix_unitaire"]

# jointure entre 2 DataFrames (équivalent d'un JOIN)
df = df_ventes.merge(df_produits, on="produit_id", how="left")

# agrégation en pandas (si on ne l'a pas faite en SQL)
df_agg = (
    df_ventes
    .groupby(["date_key", "magasin_id", "produit_id"], as_index=False)
    .agg(quantite=("quantite", "sum"),
         montant_brut=("montant_brut", "sum"),
         nb_commandes=("commande_id", "nunique"))
)
```

Écrire des données (Load)

Avec le query builder : `insert`

On transforme le DataFrame en **liste de dictionnaires**, puis un seul `insert` pour toutes les lignes :

```
from sqlalchemy import insert

# NaN -> None (sinon SQL Server reçoit "nan")
df_dim_produit = df_dim_produit.astype(object).where(df_dim_produit.notna(), None)

lignes = df_dim_produit.to_dict(orient="records")
# [{'produit_id': 1, 'code': 'GL-VAN', 'nom': 'Cornet vanille', ...}, {...}, ...]

with session_dw() as session:
    session.execute(insert(DimProduit), lignes)
    session.commit()
```

- Les clés des dictionnaires doivent correspondre aux **attributs** de la classe.
- On n'envoie **pas** la clé IDENTITY (`produit_key`) : SQL Server la génère.

Supprimer : `delete`

Pour pouvoir relancer l'ETL sans doublons, on vide les tables avant de recharger :

```
from sqlalchemy import delete

with session_dw() as session:
    session.execute(delete(FaitVente))     # d'abord les faits (clés étrangères !)
    session.execute(delete(DimProduit))
    session.execute(delete(DimMagasin))
    session.commit()
```

Tout dans une même transaction

Si une étape plante, rien n'est enregistré (pas de DW à moitié rempli) :

```
with session_dw() as session:
    session.execute(delete(FaitVente))
    session.execute(insert(FaitVente), lignes_faits)
    session.commit()      # tout ou rien
```

Alternative : `to_sql` (pandas)

pandas sait aussi écrire directement un DataFrame :

```
df_dim_produit.to_sql("dim_produit", engine_dw, if_exists="append", index=False)
```

`if_exists`	**effet**
`"fail"` (défaut)	erreur si la table existe
`"append"`	ajoute les lignes → **à utiliser pour le DW**
`"replace"`	**supprime la table** et la recrée : on perd la clé primaire, les FK, l'IDENTITY, les types !

Récupérer les clés de substitution

Après avoir chargé une dimension, on relit la correspondance **clé métier → clé de substitution** avec le query builder, puis on fait un `merge` pour préparer la table de faits :

```
df_keys = pd.read_sql(select(DimProduit.produit_key, DimProduit.produit_id),
engine_dw)

df_faits = df_agg.merge(df_keys, on="produit_id", how="left")

# contrôle : aucune ligne ne doit rester sans clé
```

```
assert df_faits["produit_key"].notna().all()

df_faits = df_faits.drop(columns=["produit_id"])
```

Rendre l'ETL relançable (idempotent)

Relancer le script ne doit pas créer de doublons. Deux approches :

1. **Tout vider puis tout recharger** (`delete` puis `insert`, voir plus haut) : simple, parfait pour un petit DW.
2. **N'insérer que les nouvelles lignes** :

```
df_existants = pd.read_sql(select(DimProduit.produit_id), engine_dw)

df_nouveaux =
df_dim_produit[~df_dim_produit["produit_id"].isin(df_existants["produit_id"])]
```

Pièges fréquents

- **Types** : vérifier avec `df.dtypes`. Une colonne d'entiers avec des NULL devient `float64` en pandas → `df["col"].astype("Int64")` (avec un I majuscule) garde les entiers.
- **NaN** : à remplacer par `None` avant un `insert` du query builder.
- **Décimaux** : arrondir avant d'insérer (`df["montant_net"].round(2)`).
- **Noms de colonnes** : le DataFrame doit avoir exactement les noms attendus par la classe / la table. Une colonne en trop → erreur.
- **Lenteur** : penser à `fast_executemany=True` sur le moteur du DW.